BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**Technical Documentation**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2026-03-01	S. Köhne	Stable	Initial Release: ProKM Kernel Architecture (Win/Linux)

1 Product Overview: ProKM (Process Kernel Module)

ProKM is the most profound security module within the ProSuite Industrial ecosystem. While conventional cryptographic solutions operate in user-space (Ring-3) and are therefore susceptible to memory dumps, side-channel attacks, and privileged malware, ProKM shifts the entire cryptographic processing (ProEDC, ProHash, ProKey) directly into the **operating system kernel (Ring-0)**.

Core Architectural Features:

- **Isolated Execution:** Key material and plaintext data never leave the kernel-space.
- **Cross-Platform API:** Unified IOCTL interfaces for Windows (KMDF) and Linux (Character Device).
- **Zero-Trust Memory Management:** Guaranteed physical RAM wiping before memory deallocation.

2 System Limits & Memory Management

Since operations in kernel-space can have critical impacts on system stability (Kernel Panics / BSODs), ProKM implements strict security boundaries.

2.1 The 32-Megabyte Transaction Limit

To prevent memory fragmentation and paging issues within the kernel, every single IOCTL transaction (encryption/decryption or hashing) is strictly limited to exactly **32 Megabytes (`PROKM_MAX_ALLOC`)**.

- **Large Files:** If files larger than 32 MB need to be processed, the user-space application (e.g., the C# wrapper) must chunk the data into blocks of up to 32 MB and send them sequentially to the driver.

2.2 Memory Allocation & Secure Wipe

- **Windows:** The driver utilizes `ExAllocatePoolWithTag(NonPagedPoolNx, ...)`. This ensures the allocated memory is protected against execution (No-Execute) and cannot be paged out to the disk (Pagefile). After the operation, the memory is thoroughly overwritten using `RtlSecureZeroMemory`.

- **Linux:** `vmalloc()` is used to securely allocate large, contiguous virtual memory blocks. The cleanup is performed via `memzero_explicit()` to prevent the compiler from optimizing away the wiping process.

3 IOCTL Interfaces (Driver Calls)

Communication between the user-space application and the ProKM driver is handled asynchronously via standardized IOCTL calls. The driver's magic number is `'k'`.

3.1 Supported Commands (Command Codes)

Command	Windows (CTL_CODE)	Linux (_IOWR)	Description
`IOCTL_PROKM_PROED`	`0x801`	`1`	Executes encryption or decryption (AES/ProEDC).
`IOCTL_PROKM_PROHASH`	`0x802`	`2`	Calculates the 256-bit integrity hash.
`IOCTL_PROKM_PROKEY`	`0x803`	`3`	Requests hardware entropy (randomness).
`IOCTL_PROKM_STATUS`	`0x899`	`99`	Retrieves the health status of the driver.

3.2 Data Structures (ABI-Safe)

The memory structures are packed using `#pragma pack(push, 1)` to prevent cross-platform padding issues between 32-bit and 64-bit systems.

1. ProED Encryption Request (`ProKM_ProED_Req`) This structure is used for `IOCTL_PROKM_PROED`. The buffer must be large enough to hold the header and the variable `payload`.

```
typedef struct {
    uint8_t  key[32];      // 256-bit Master Key
    uint8_t  iv[16];       // 128-bit Initialization Vector
    uint32_t mode;         // 0 = Encrypt, 1 = Decrypt
    uint32_t data_len;     // Length of the payload (Max 32 MB)
    uint8_t  payload[1];   // Start of the variable data stream
} ProKM_ProED_Req;
```

2. ProHash Integrity Request (`ProKM_ProHash_Req`) Used for `IOCTL_PROKM_PROHASH`. The kernel returns the original payload data *and* the 32-byte hash at the end of the payload buffer.

```
typedef struct {
    uint32_t data_len;     // Length of the data to be hashed
    uint8_t  payload[1];   // Data stream
} ProKM_ProHash_Req;
```

4 Operating System Integration

4.1 Windows (KMDF)

On Windows, the driver is loaded as `ProKM_Driver.sys`.

- **Device Path:** Applications (e.g., in C#) must open a file handle to `\\.\ProKM` (`\DosDevices\ProKM`) to communicate with the kernel via `DeviceIoControl()`.
- **Security:** The driver verifies itself upon loading via the internal `ProKM_RunSelfTest()`. If this fails (POST FAILED), the driver refuses to start (`STATUS_DEVICE_CONFIGURATION_ERROR`).

4.2 Linux (Character Device)

On Linux, the driver is installed as a loadable kernel module (`prokm.ko`).

- **Device Path:** A node is created under `/dev/prokm`. User-space programs open this node using `open()` and send commands via `ioctl()`.
- **Memory Protection:** The driver strictly uses `copy_from_user` and `copy_to_user` to prevent memory access violations (segfaults) in case malicious or invalid pointers are passed from user-space.

5 Error Codes & Troubleshooting

If an IOCTL call fails, the driver returns system-specific error codes:

- **Windows `STATUS_INVALID_PARAMETER` / Linux `-EINVAL`:** Triggered if `data_len == 0` or if the hardcoded limit of 32 MB (`PROKM_MAX_ALLOC`) is exceeded.
- **Windows `STATUS_BUFFER_TOO_SMALL` / Linux `-EFAULT`:** The provided user-space buffer is smaller than specified in `data_len` or not large enough to hold the response data.
- **Windows `STATUS_INSUFFICIENT_RESOURCES` / Linux `-ENOMEM`:** Critical system state. The kernel does not have enough free memory available for the allocation (`NonPagedPoolNx` / `vmalloc`).

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**